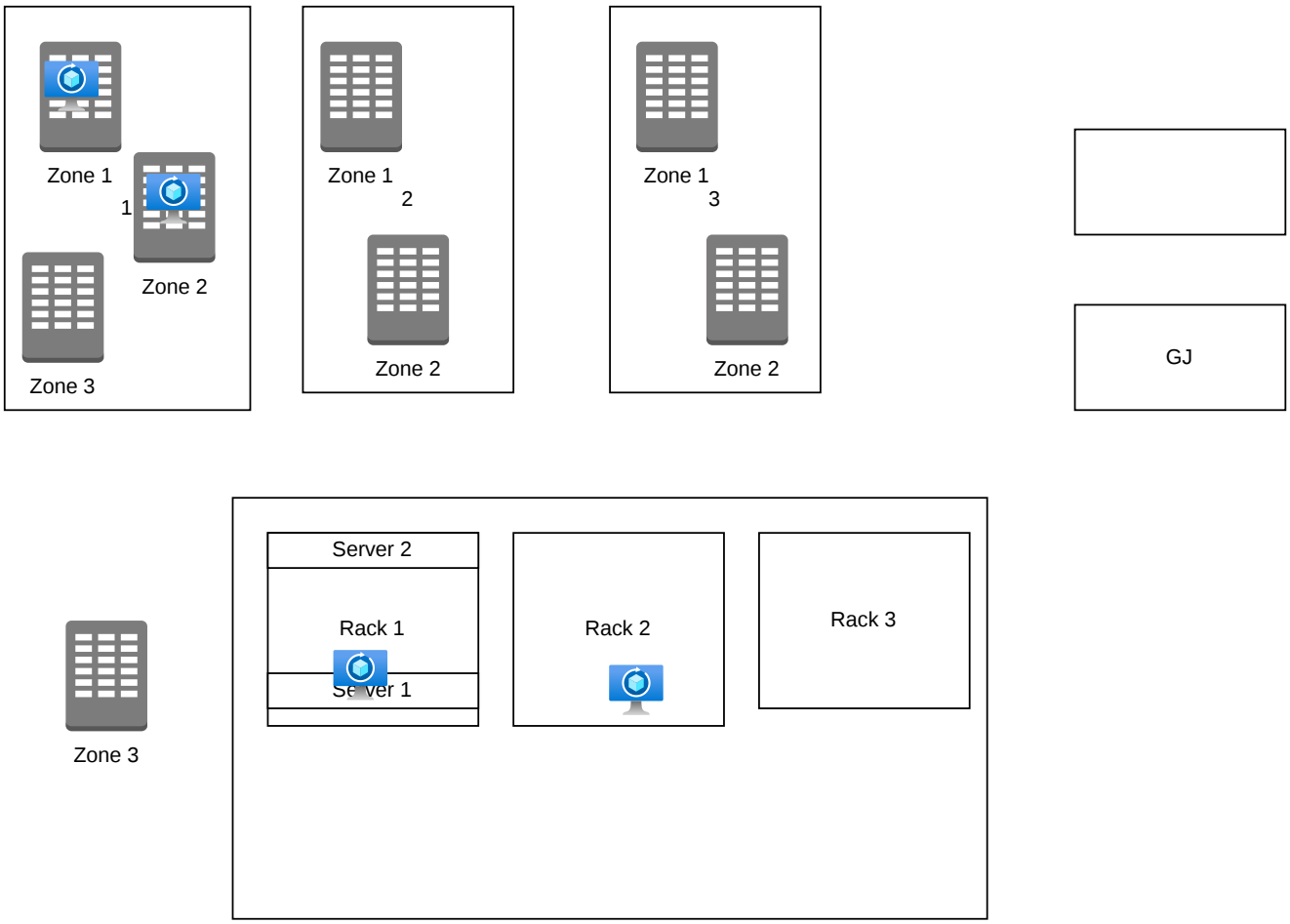
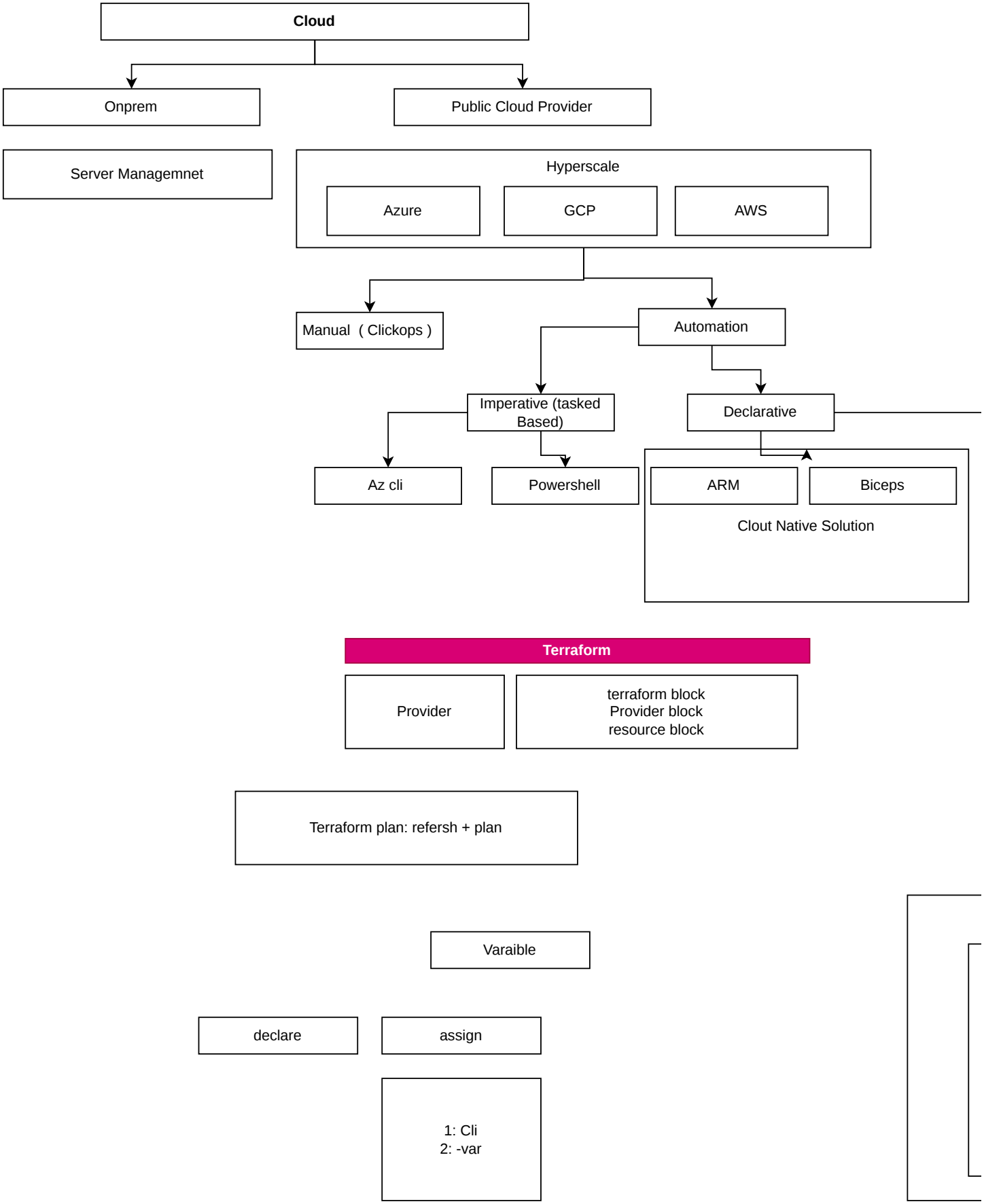
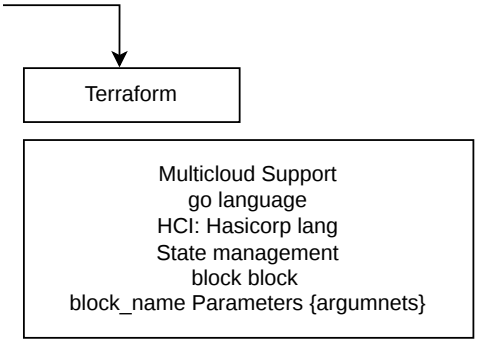
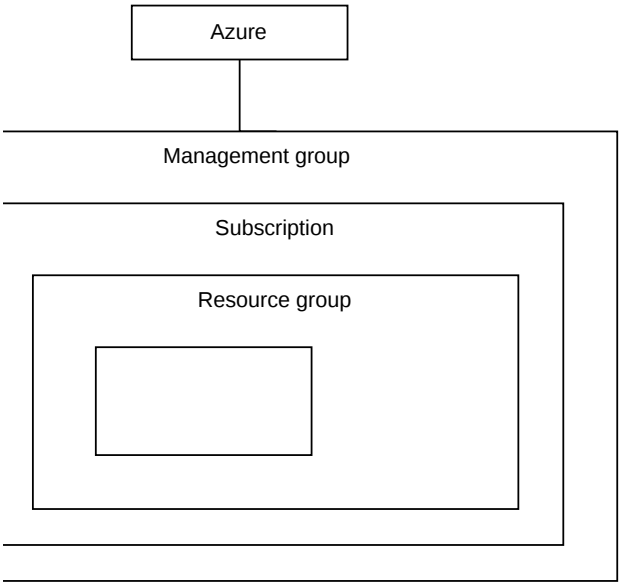

Zone 1
1
Zone 2
Zone 3
Zone 1
2
Zone 2
Zone 1
3
Zone 2
GJ
Zone 3
Server 2
Rack 1
Server 1
Rack 2
Rack 3


Cloud
Onprem
Public Cloud Provider
Server Managemnet
Hyperscale
Azure
GCP
AWS
Manual (Clickops)
Automation
Imperative (tasked Based)
Declarative
Az cli
Powershell
ARM
Biceps
Clout Native Solution
Terraform
Provider
terraform block
Provider block
resource block
Terraform plan: refersh + plan
Varaible
declare
assign
1: Cli
2: -var


Terraform
Multicloud Support
go language
HCI: Hasicorp lang
State management
block block
block_name Parameters {argumnets}


Azure
Management group
Subscription
Resource group

Vnet, subnet, nic, basion, nsg

Virtual Network: Logical network seperation

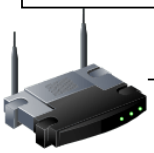
Subnet

NIC

RAM/CPU, DISK, NIC

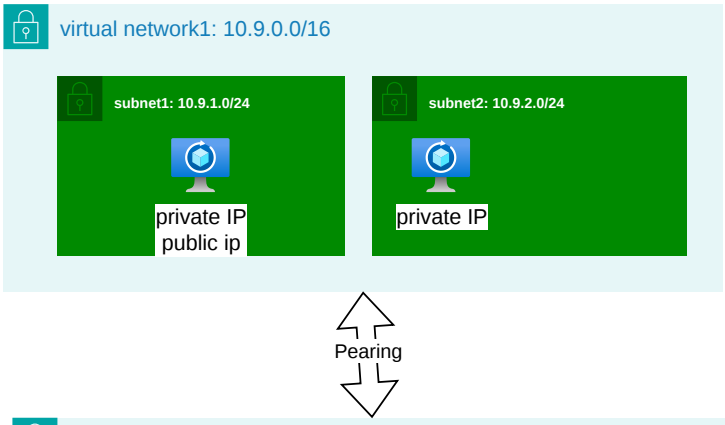
Public IP

Private IP



251 IP's

CIDR: IP/0-32: 2 ki power n - 32
10.9.0.0/24 = > 2^24-32 = 2^8 = 256



VM

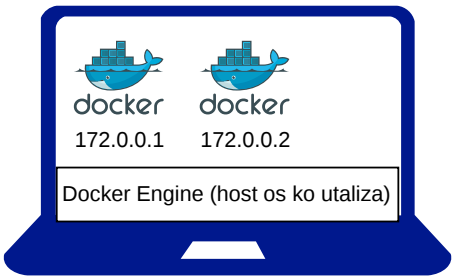
azurerm_resource_group
azurerm_virtual_network
azurerm_subnet
azurerm_network_interface
azurerm_virtual_machine

PORT

Home

0-65000+

13 and 14th Class revision

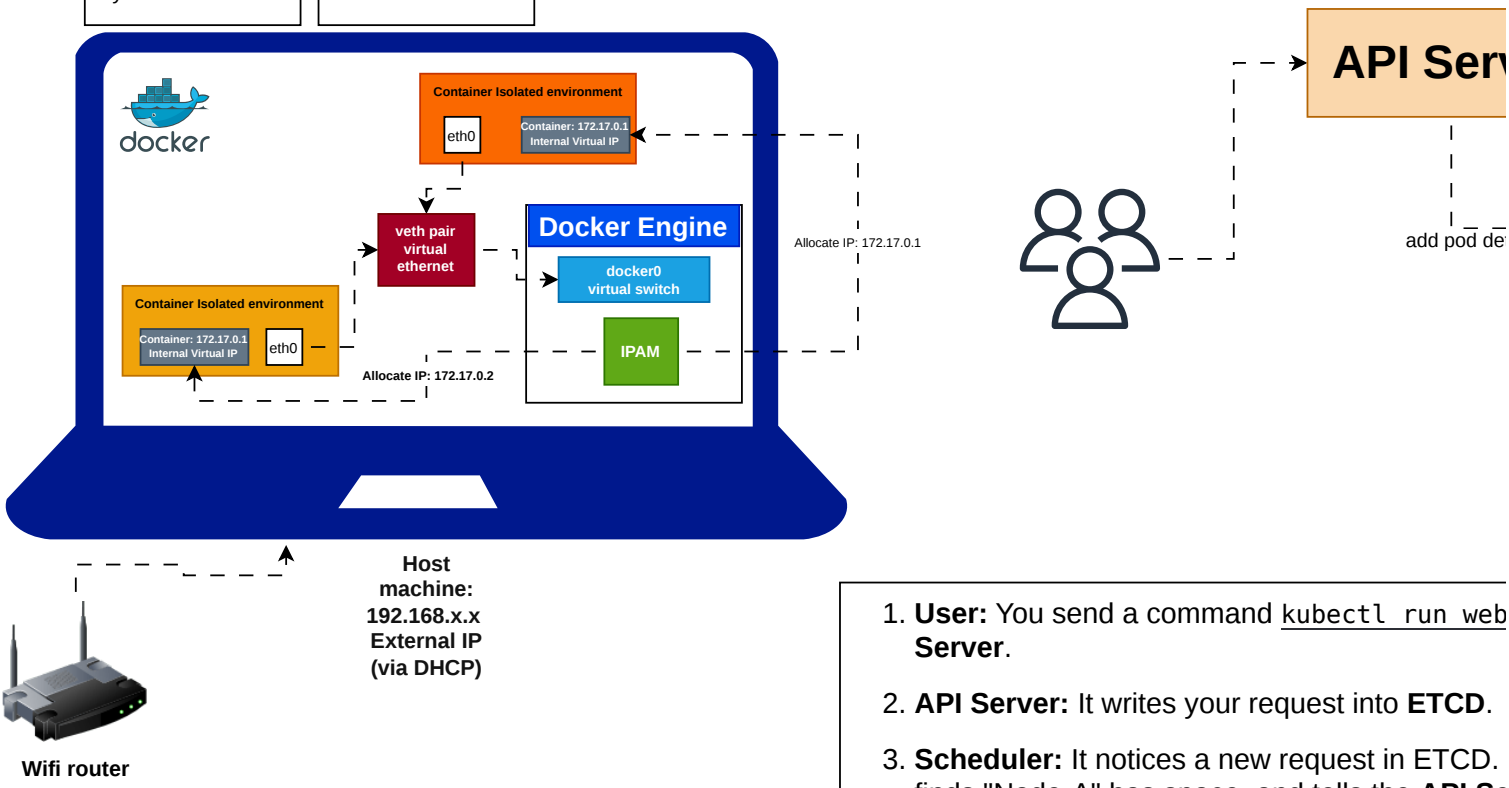


In Docker: each container get the ip address

VM	Docker
VM has own Operating system file system system commands	System commands file system host os utalization

K8s Revision

Master	Workers
1: API Server 2: etcd 3: scheduler 4: controller-manager	1: kubelet 2: kube proxy 3: container runtime



Host machine: 192.168.x.x External IP (via DHCP)

Wifi router

1. **User:** You send a command `kubectl run web` **Server**.
2. **API Server:** It writes your request into **ETCD**.
3. **Scheduler:** It notices a new request in ETCD. finds "Node-A" has space, and tells the **API Se** on Node-A."
4. **API Server:** Updates **ETCD** with this plan.
5. **Kubelet (on Node-A):** It's constantly listening sees: "Oh, I have a new task!"
6. **Container Runtime: Kubelet** tells the **Runtim** app image and start it."
7. **Kube-Proxy:** Updates the networking rules so visit your website.
8. **Controller Manager:** Keeps watching to make never stays down.

- **API Server (The Receptionist):**
- This is the only "entry point." Every command you give (like `kubectl apply`) goes here.
- **Role:** It validates your request and passes it to the other components. It's the "Front Desk" of the

ver

ack POD == Pedning

tails

etcd

Text

Scheduler

- **Scheduler (The Resource Plar**
-
  - When you want to start a new
  - **Role:** It checks which node ha manager deciding which work

Controller Manager

Controller Manager (The Watchm

- It constantly compares the **Cu**
- **Role:** If you asked for 3 conta and tells the API server to sta

Kubelet

Kubelet (The Supervisor):

- It is a small agent running on
- **Role:** It watches the API Serv and healthy. If a container sto
- *Analogy:* The supervisor on th

Container Runtime

Container Runtime (The Engine -

- Kubelet doesn't "run" containe
- **Role:** It pulls the image and s

`-app` to the **API**

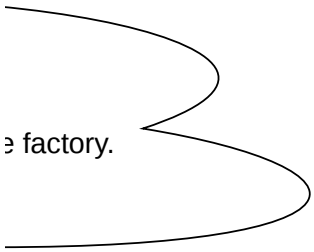
it looks at the Nodes, **erver**: "Put web-app

to the API Server. It

ne: "Hey, pull the web-

users can actually

e sure that web-app



nner):

container, the Scheduler looks at all the Worker Nodes.

as enough RAM/CPU and "assigns" the task to the best-fit node. It's like a ker is free to take a shift.

an):

urrent State with the **Desired State**.

ainers but one crashes, the Controller Manager notices the count is now 2 rt a new one. It handles the "Self-healing."

every node.

ver for tasks assigned to its node. It makes sure the containers are running ps, Kubelet tries to restart it.

he factory floor making sure workers are actually working.

e.g., containerd/Docker):

ers; it tells the Runtime to do it.

starts/stops the container. It's the actual "machine" that builds the product.

1. Container-to-Container (The Pod leve

Ek Pod ke andar ek se zyada containers ho sakte ha

- **Concept:** Ye sabhi containers ek hi **Network N**
- **Kaise hota hai?** Ye sabhi `localhost` par ek du 80 par hai aur dusra Port 5000 par, toh wo dire

3. Pod-to-Service (Stable Identity)

Pods temporary hote hain (wo delete aur restart hote **Service** use karte hain.

- **ClusterIP:** Ye ek virtual IP hota hai jo Pods ke
- **Kube-Proxy:** Har Node par ek component hot IPTables or IPVS).
- **Service Discovery (CoreDNS):** Aapko IP yaa (e.g., `my-db-service`) use karte hain, aur Kube

5. Network Policies (The Security Layer

Enterprise level par security ke liye hum Network Po

- **Concept:** By default, saare Pods aapas mein I
- **Control:** Isse aap restrict kar sakte hain ki "Sir Database se nahi." Ye firewall rules ki tarah ka

IP Ma

Chapter 1: Pod IP kaise milta hai? (The

Kubernetes mein har Pod ko ek real IP milta hai. Yeh karta hai.

- **IP Address Management (IPAM):** Jab aap kul bolta hai, *"Bhai, ek naya Pod aa raha hai, isse*
- **Veth Pairs (The Pipe):** GNU do "virtual Ethern

Kubernetes networking

el)

ain.

Namespace share karte hain.

usre se baat kar sakte hain. Jaise ek container Port
ect connect ho jayenge.

2. Pod-to-Pod Networking (The Cluster level)

Kubernetes ka sabse bada rule hai: **Har Pod ka apna ek Unique**
par ho.

- **CNI (Container Network Interface):** Kubernetes khud netv
(jaise **Calico, Flannel, ya Azure CNI**) par depend karta hai
- **Bina NAT ke baat:** Ek Pod dusre Pod ko uske IP se directly
Mapping ke.
- **Overlay Network:** CNI ek virtual jaal (tunnel) banata hai tal
aise baat kare jaise wo ek hi switch se jude honge.

e rehte hain aur IP badalta rehta hai). Isliye hum

group ko milta hai.

a hai jo traffic ko sahi Pod tak redirect karta hai (using

d rakhne ki zaroorat nahi. Aap sirf service ka naam
ernetes DNS use resolve kar deta hai.

4. External-to-Service (Ingress & LoadBalancer)

Jab cluster ke bahar se (Internet se) traffic andar aata hai:

- **NodePort:** VM ka ek specific port (30000-32767) khol deta
- **LoadBalancer:** Cloud provider (Azure/AWS) ka ek external
cluster mein bhejta hai.
- **Ingress Controller:** Ye ek **Layer 7 (HTTP/HTTPS)** gatekee
`://example.com` vs `://example.com`) traffic ko alag-alag se

r)

olicies use karte hain.

baat kar sakte hain.

f Frontend Pod hi Backend Pod se baat kare,
am karte hain.

Summary Sequence:

1. **Traffic Internet se aaya** → **Ingress/LoadBalancer** ne pak
2. **Service** (ClusterIP) ne decide kiya kis Pod group ke paas ja
3. **Kube-Proxy** ne traffic ko sahi **Pod IP** par bheja.
4. **CNI** ne traffic ko sahi **Node** aur sahi **Container** tak deliver k

nagement (CNI), Traffic Routing (Kube-Proxy), aur Name Resolution (CoreDNS).

CNI Layer)

h kaam **CNI (Container Network Interface)** plugin

`bectl apply` karte hain, toh Kubelet CNI plugin ko
IP do."

Chapter 2: Service kaam kaise karti hai? (Kube

Service ka koi physical existence nahi hota; yeh sirf ek **Virtual**
Isse manage karta hai **Kube-Proxy**.

- **Service Creation:** Jab aap ek Service banate hain, toh
(ClusterIP).

e **IP hota hai**, chahe wo kisi bhi Node

working nahi karta, wo **CNI Plugins**
i.

y reach kar sakta hai, bina kisi Port

ki Node A ka Pod, Node B ke Pod se

hai.

l Load Balancer banata hai jo traffic ko

eper hai. Ye rules ke hisab se (e.g.,
ervices tak pahunchata hai.

da.

ana hai.

kiya

e-Proxy & IP Tables)

l **IP (VIP)** hai jo kabhi change nahi hota.

Control Plane usse ek IP deta hai

- **veth Pairs (The Pipe):** CNI do "Virtual Etherne
  - Ek end Pod ke andar hota hai (`eth0`).
  - Doosra end Node ke Network Bridge (`cb`
- **The Result:** Is pipe ki wajah se Pod ko lagta h Nodes ke Pods se baat kar pata hai.

Chapter 3: DNS aur Service Discovery (

Aap Pods ko IP se nahi, naam se dhundte hain.

- **CoreDNS:** Yeh cluster ke andar ek chhota sa s
- **The Process:**
  1. Frontend Pod bolta hai: *"Mujhe `order-db`*
  2. Woh request CoreDNS ke paas jati hai.
  3. CoreDNS reply karta hai: *"Order-DB ka S*
  4. Ab Frontend Pod us IP par traffic bhejta h hain.

3. Azure CNI vs Kubenet (Implementati

Enterprise level par aapko yeh chunna pa

Feature	Kubenet (Basic)
IP Kahan se milta hai?	Ek alag internal r
Visibility	Pod IP VNET ke l chahiye).
Complexity	Simple, kam IP la
Speed	Thodi slow (NAT

Deep Dive: Step-by-Step Process

Jab aap `kubectl apply` karte hain, toh yeh

Step 1: Network Namespace Creation
Kubelet sabse pehle Pod ke liye ek alag **Netw** (NIC) hai.

Step 2: veth-pair Generation
Kubelet **CNI Plugin** (jaise Calico ya Azure CNI

et" (veth) wires banata hai.

`r0` ya `docker0`) se juda hota hai.

ai ki woh physical network se juda hai, aur woh doosre

- **The Magic of IP Tables:** Kube-Proxy har Node par baith hai, woh Node ke Kernel mein **IP Tables** (Linux firewall r
  - *Rule:* "Agar koi traffic `10.0.0.1:80` (Service IP) pa ya `192.168.1.6` (Pod IPs) par bhej do."
- **Load Balancing:** Yeh "Random" distribution hi Kubernet

(CoreDNS)

server hai jo har Service ka record rakhta hai.

se baat karni hai."

Service IP `10.96.0.10` hai."

hai, aur IP Tables usse sahi Pod tak pahuncha dete

Chapter 4: Ingress vs LoadBalancer (External

Jab traffic Internet se aata hai, toh rasta thoda lamba hota hai:

1. **Cloud Load Balancer:** Azure/AWS ek Public IP deta hai
2. **Ingress Controller (e.g., Nginx):** Yeh ek "Smart Gateke
   - Kya request `/api` ke liye hai? Toh API service ko d
   - Kya request `/login` ke liye hai? Toh Auth service k
3. **SSL Termination:** Ingress hi woh jagah hai jahan aap a

on Difference)

dta hai ki IP kaise manage honge:

	Azure CNI (Advanced)
ange se.	Aapke **VNET** ke subnet se (Directly).
bahar nahi dikhta (NAT	Pod IP pure office network mein dikhta hai.
gte hain.	Thoda complex, har Pod ke liye ek VNET IP reserve hota hai.
ki wajah se).	Sabse fast (Direct communication).

5 steps hote hain:

ork Namespace banata hai. Yeh ek isolated boundary hai taki Pod ko lage ki uske paas apna khud ka netw

I) ko call karta hai. CNI ek **veth-pair** (Virtual Ethernet pair) create karta hai.

ıa hota hai. Jaise hi naya Service banta
ules) likh deta hai:

r aaye, toh usse randomly `192.168.1.5`

tes ka basic load balancing hai.

Traffic)

:

i.

eper" hai. Yeh check karta hai:

o.

o do.

pne SSL certificates (HTTPS) rakhte hain.

vork card

- Yeh ek aisi wire hai ki agar aap ek end pa

Step 3: Plugging the Wires

- Wire ka **ek end** Pod ke namespace ke an
- Wire ka **doosra end** Node ke main netwo

Step 4: IP Address Assignment (IPAM)
CNI ke paas ek **IPAM (IP Address Manageme**

- Yeh check karta hai ki Cluster ke liye allot
- Woh ek IP (e.g., `10.244.1.5`) uthata ha

Step 5: Routing Update
CNI Node ke routing table ko update karta hai
hai.

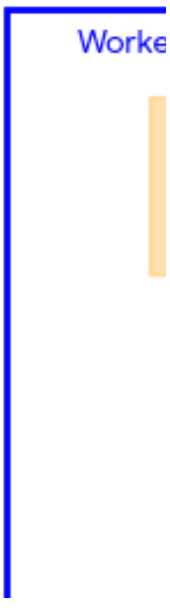
Intra-Node Networking (Ek hi Node ke d

- **The Flow:** Agar Pod A aur Pod B ek hi Node p
- Woh Node ke andar bane **Virtual Bridge** (Swit

Inter-Node Networking (Alag-alag Node

Yeh sabse important part hai. Jab Node 1 ka Pod, N

- **Encapsulation (Overlay):** CNI ek packet bana
- **The Route:** Traffic Node 1 se nikalta hai, physi
- **Decapsulation:** Node 2 ka CNI envelop fadta



ar data dalenge, toh woh doosre end se nikal jayega.

ndar dala jata hai aur uska naam badal kar `eth0` rakh diya jata hai.

ork (Root Namespace) mein rakha jata hai aur use Node ke **Bridge** se connect kar diya jata hai.

ent) plugin hota hai.

tted IP range (CIDR) mein se kaunsa IP khali hai.

i aur use Pod ke `eth0` interface ko assign kar deta hai.

taki jab koi traffic us specific IP (`10.244.1.5`) ke liye aaye, toh Node ko pata ho ki use kis "wire" (veth) me

---

---

do Pods)

ar hain, toh unka traffic Control Plane tak nahi jata.

tch) ke zariye aapas mein baat kar lete hain. Yeh bilkul waisa hai jaise ek hi ghar ke do log aapas mein baat kar rahe ho.

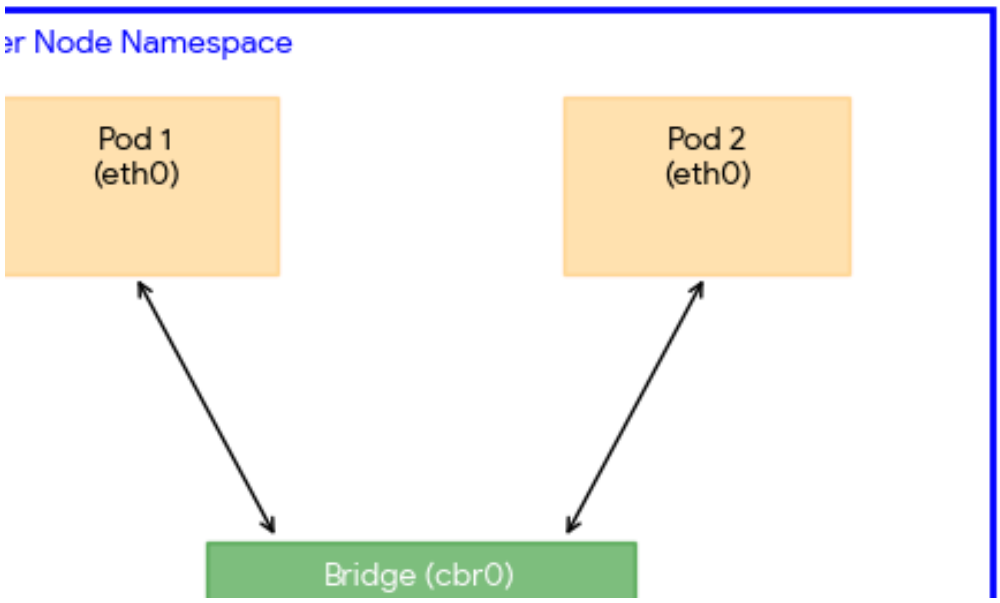
es ke Pods)

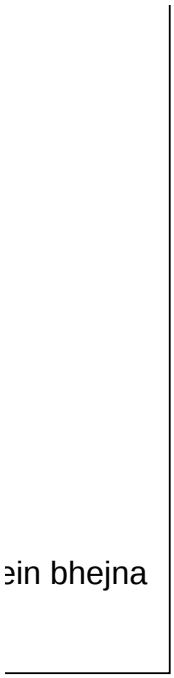
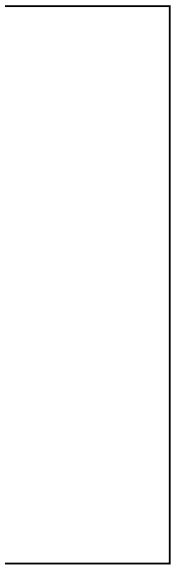
ode 2 ke Pod se baat karta hai:

ata hai jisme Pod ka IP hota hai, aur usse ek "Envelop" mein dal deta hai jiske upar **Node ka IP** likha hota hai.

ical network se hota hua Node 2 tak pahunchta hai.

hai aur asli packet Pod tak pahuncha deta hai.




ein bhejna


ClusterIP Kubernetes ka sabse basic aur default Service

1. Kyu hota hai? (The "Why")

Pods "Ephemeral" hote hain (wo kabhi bhi mar sakte hai

- Jab naya Pod banta hai, uska **IP badal jata hai**.
- Agar aapka Frontend Pod backend ke IP `10.244.1`
- Frontend crash ho jayega kyunki use purana IP pa
- **Solution:** ClusterIP ek "Vicholiya" (Middleman) hai

---

2. Kya hota hai? (The "What")

ClusterIP ek **Virtual IP** hai. Yeh koi asli Network Card (N

- Yeh sirf Cluster ke **andar** se accessible hota hai.
- Iske saath ek **Selector** hota hai (jaise `app: backer`

---

3. Kaise hota hai? (The Networking Flow)

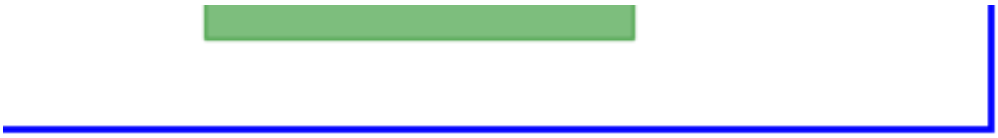
Iska flow 3 main components ke beech chalta hai: **Contr**

Step 1: Service Creation
Jab aap Service banate hain, Control Plane (API Server)

Step 2: Kube-Proxy ki Entry
Har Worker Node par **Kube-Proxy** baitha hota hai. Woh
controller) mein rules likh deta hai.

Step 3: Traffic Flow (The Real Magic)

1. **Frontend Pod** request bhejta hai: `10.100.5.5:80`
2. Traffic jaise hi Pod se nikal kar Node ke network pa
3. IP Tables ke paas rule hota hai: *"Agar koi `10.100.5`*
4. IP Tables request ka **Destination IP** badal kar asli



---

e type hai. Iska kaam hai Pods ke group ko ek **Fixed (Permanent) IP** dena.

in ya restart ho sakte hain).

`l.5` se baat kar raha hai aur backend restart ho gaya, toh uska naya IP `10.244.1.8` ho jayega.

ta hai.

i. Frontend sirf ClusterIP se baat karta hai, aur ClusterIP piche ke badalte hue Pod IPs ko handle karta hai.

IC) nahi hai, balki Linux kernel ke andar ek **Rule** hai.

nd), jo decide karta hai ki traffic kin Pods ke paas jayega.

---

rol Plane, **Kube-Proxy**, aur **IP Tables**.

) use ek IP deta hai (e.g., `10.100.5.5`). Yeh info **etcd** mein save ho jati hai.

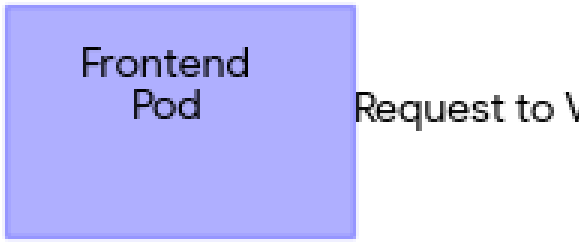
hamesha API Server ko dekhta rehta hai. Jaise hi use naya Service dikhta hai, woh Node ke **IP Tables** (Linux ka traffic

(ClusterIP).

ar jata hai, **IP Tables** use pakad leta hai.

`5.5` par aaye, toh usse 50% `Pod-A` par aur 50% `Pod-B` par bhej do."

Pod IP (`10.244.1.x`) kar deta hai. Isse **DNAT** (Destination Network Address Translation) kehte hain.



NodePort ClusterIP ka ek agla step hai. Jab a
karte hain.

1. Simple Words mein Concept

Maan lijiye aapka cluster ek "Society" hai.

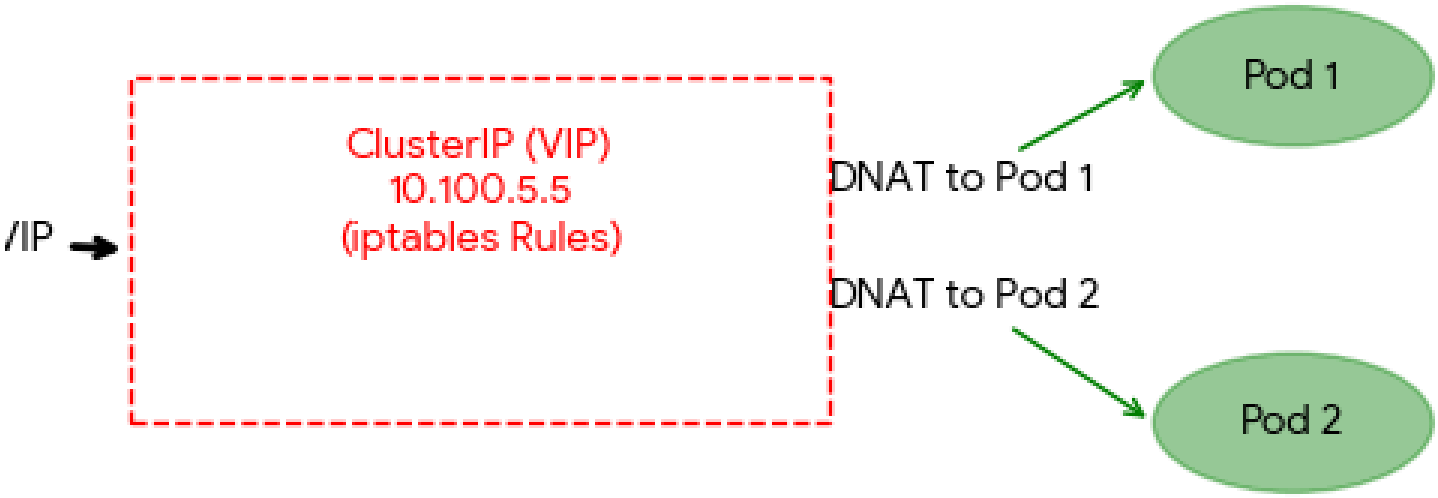
- **ClusterIP:** Society ke andar ka intercom (
- **NodePort:** Society ka "Main Gate" par ek

---

2. NodePort kaise kaam karta hai? (End-to-

Jab aap ek NodePort Service banate hain, toh

1. **Fixed Port Assignment:** Kubernetes har
2. **Open Gate on Every Node:** Yeh port clus



ıapko Kubernetes cluster ke bahar se (Internet ya Office Network se) apni application access kar

(sirf andar ke log baat kar sakte hain).

: khidki khol dena. Ab koi bhi bahar se us khidki (Port) par aakar andar ke kisi bhi flat (Pod) se ba

---

End Flow)

ı 3 cheezein hoti hain:

Worker Node par ek specific port reserve kar deta hai. Iski range hamesha **30000-32767** ke bee

ster ke **har ek node** par khul jata hai. Chahe us node par aapka Pod chal raha ho ya nahi, woh p

ni hoti hai, tab hum **NodePort** use

at kar sakta hai.

ch hoti hai.

ort traffic accept karega.

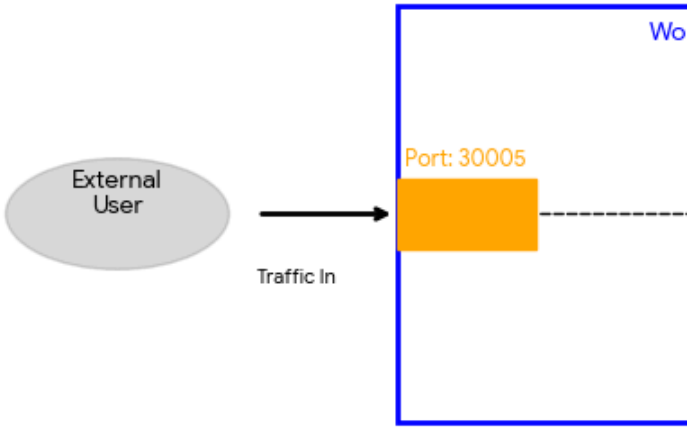
3. **Internal Routing:** Jaise hi traffic Node ke

---

3. Step-by-Step Traffic Journey (The Flow)

Maan lijiye aapka Node IP `192.168.1.10` ha

1. **User Request:** User browser mein type k
2. **Node Entry:** Traffic Worker Node par pah
3. **Kube-Proxy & IP Tables:** Node ke andar
4. **Load Balancing:** ClusterIP ab dekhta hai
5. **Pod Delivery:** Traffic final **Pod IP** par pah



Load Balancer service type Kubern
Provider (jaise Azure, AWS, Google

1. Yeh hota kya hai? (The Concept

us port par aata hai, woh use automatically internal **ClusterIP** par bhej deta hai, jo aage Pod tak

---

ı aur NodePort `30005` hai.

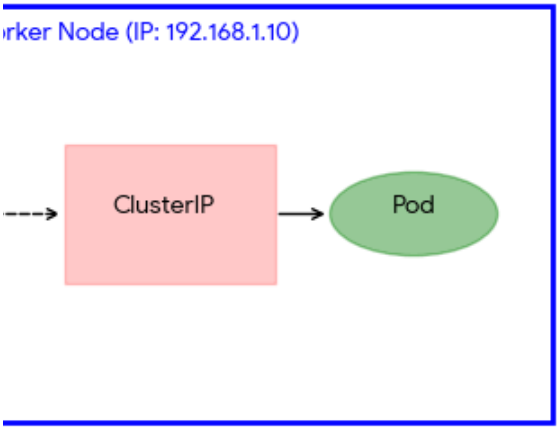
arta hai `http://192.168.1.10:30005`.

unchta hai. Node dekhta hai ki port `30005` par request aayi hai.

ka Kube-Proxy/IP Tables rule ise pehchan leta hai. Woh request ko "Internal ClusterIP" ki taraf m

ki piche kaun-kaun se Pods (`Selector` ke base par) available hain.

unch jata hai.



5. Why and When to use (and when NOT

- **Pros:** Setup karna bahut asan hai aur perfect hai.
- **Cons:**
  - Security risk hai kyunki ports direct
  - Agar Node ka IP badal gaya, toh a denge.
  - Ek port par sirf ek hi service chal s

Enterprise Level par kya karte hain?
Real companies NodePort ko directly use r **Load Balancer** lagati hain jo saare Nodes manage karta hai.

etes mein networking ka "VIP Entrance" hai. Yeh as mein **NodePort** ke up Cloud) ke sath mil kar kaam karti hai.

t)

‹ le jata hai.

nod (redirect) deta hai.

Γ to)

demo/testing ke liye

ly open ho rahe hain.

ap connection kho

akti hai.

nahi karti. Iske upar ek
ke IPs aur Ports ko

ar ek extra layer hai jo Cloud

Maan lijiye aapke pass 3 Worker No

- **Problem:** User ko kaunsa Node
- **Solution:** Cloud provider aapko e
  hai, aur wahan se Nodes mein di

---

2. End-to-End Traffic Flow (Step-b

Jab aap `type: LoadBalancer` wa

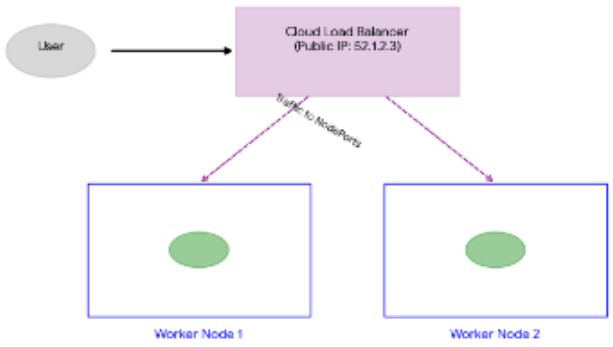
1. **Service Creation:** Kubernetes A
2. **Cloud Action:** Azure ek **Standar**
3. **Automatic NodePort:** Kubernete
4. **Health Checks:** Load Balancer N
   deta hai.

Real Traffic Journey:

- **User:** `http://52.1.2.3:80` (F
- **Cloud LB:** Traffic pakadta hai au
- **Worker Node:** Kube-proxy/IP Tal
- **Pod:** Finally, traffic aapke applica

---

3. Load Balancer Architecture (Dia



des hain aur sab par NodePort khula hai.

IP dena hai? Agar woh Node down ho gaya toh?

ek **Single Public IP** (e.g., `52.1.2.3`) deta hai. Yeh IP kabhi nahi badalta.
vide ho jata hai.

---

y-Step)

ali service deploy karte hain, toh background mein yeh steps hote hain:

PI Server Cloud Provider (e.g., Azure) ko signal bhejta hai: *"Mujhe ek Exte*

rd Load Balancer resource banata hai aur usse ek Public IP assign karta

es piche se automatically ek **NodePort** bhi kholta hai (taaki Load Balancer

Nodes ko "Ping" karke check karta rehta hai. Agar koi Node dead hai, toh v

Public IP) par click karta hai.

r use kisi bhi healthy Worker Node ke **NodePort** (e.g., 31050) par bhej de

bles traffic ko internal **ClusterIP** par bhejte hain.

ation Pod tak pahunch jata hai.

---

agram)

Traffic pehle is Public IP par aata

ernal Load Balancer chahiye."

hai.

us port par traffic bhej sake).

woh wahan traffic bhejna band kar

ta hai.

4. Enterprise Problem: Load Balan

Har Service ke liye ek naya Load Ba

- **Cost Badhti Hai:** Azure har Publ
- **Management Mushkil:** Agar aap

Solution: Iske baad hum **Ingress** us

Ingress Controller Kubernetes networking ka sabse "intelli
dekh kar tay karti hai ki aapko kis kamre (Service) mein bhe

1. Ingress ki Zarurat Kyu Padti Hai? (The Pro

Pichle step mein humne dekha ki har `LoadBalancer` service

- **Problem:** Agar aapki 10 microservices hain (`/api`, `/l`
- **Solution:** Aap sirf **EK Load Balancer** lagate hain aur

---

2. Ingress kaise kaam karta hai? (End-to-End

Iske do main parts hote hain:

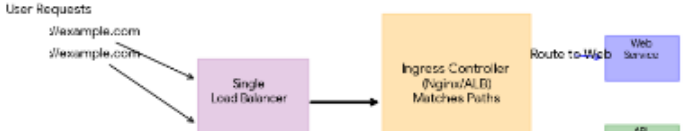
1. **Ingress Resource:** Yeh ek Rule book hai (YAML file)
   [1]
2. **Ingress Controller:** Yeh woh software hai (jaise **Ngin**

Traffic ka Safar (The Journey):

1. **User Request:** User browser mein dalta hai `://examp`
2. **Cloud Load Balancer:** Traffic ek hi Public IP par aata
3. **Rule Matching:** Ingress Controller request ka **Host H**
4. **Forwarding:** Woh traffic ko direct backend **Pods** tak
   performance ke liye).

---

3. Ingress Architecture (Diagram)



ncer Mehenga Kyu Hai?

lancer banane se:

lic IP aur LB ka paisa leta hai.

oke pass 50 microservices hain, toh 50 Load Balancers manage karna sar

se karte hain, jo **ek hi Load Balancer** par multiple services chalane ki perr

igent" hissa hai. Agar Load Balancer ek simple gatekeeper hai, toh Ingress ek **Smart Receptionist** hai jo URL ejna hai.

oblem)

e ek naya Public IP aur naya paisa kharch karti hai.

ogin`, `/orders`), toh kya aap 10 alag Load Balancers kharidenge? Yeh bahut mehenga padega.

uske piche **Ingress Controller** bitha dete hain.

---

d Flow)

jisme aap likhte hain: *"Agar traffic `/api` par aaye toh Service A ko do, aur `/web` par aaye toh Service B ko do."*

nx, Traefik, ya Azure Application Gateway) jo un rules ko asaliyat mein execute karta hai. [1, 2]

ple.com`.

a hai aur use **Ingress Controller Pod** ke paas bhej deta hai.

leader (`example.com`) aur **Path** (`/api`) check karta hai.

pahuncha deta hai (aksar yeh ClusterIP service ko bypass karke direct Pod IPs par traffic bhejta hai

---

dard hai.

mission deta hai.

4. Ingress ke Khass Features (Enterprise Lev

- **SSL Termination:** Aapko har Pod mein Certificate da
- **Path-Based Routing:** `/app1` ko Service 1 par bhejo, /
- **Host-Based Routing:** `blog.com` ko Blog service par b

🏆 End-to-End Networking Summary:

1. **Pod IP (CNI):** Container ko address deta hai.
2. **ClusterIP:** Pods ko stable identity deta hai (Internal).
3. **NodePort:** External traffic ke liye rasta kholta hai (Bas
4. **LoadBalancer:** External traffic ko Public IP aur reliabi
5. **Ingress:** Ek hi IP par multiple services ko URL ke hisa

vel)

lne ki zarurat nahi. Ingress par ek baar SSL certificate laga do, saari services HTTPS ho jayengi.

`/app2` ko Service 2 par.

bhejo aur `shop.com` ko Shop service par (ek hi IP use karke).

sic).

ility deta hai.

ab se manage karta hai

Virtual Machine (VM)

VM ek **"Managed Disk"** mein store hoti hai, jo bilkul aapke laptop ki **Hard Drive** ki tarah kaam karti hai. Azure ise "Storage Account" ke andar **Page Blob** format mein save karta hai.

VM ka size kaise increase (resize) karte hain?

Agar aapki VM slow chal rahi hai aur aap uski **RAM ya CPU** badhana chahte hain, toh ye steps follow karein:

1. **VM Stop Karein:** Sabse pehle Azure Portal par ja kar apni VM ko **Stop (Deallocate)** karein. (Chalti VM ka size change nahi hota).
2. **Size Par Jayein:** VM ki settings (left menu) mein **Size** par click karein.
3. **Naya Size Chunein:** List mein se bade size (zyada RAM/vCPU) ko select karein.
4. **Resize Karein:** Niche diye gaye **Resize** button par click karein.

Enterprise level par Azure VM access na hone ke pichhe kai networking aur security reasons ho sakte hain. Iske main 5 reasons niche diye gaye hain:

1. Network Security Group (NSG) Rules

Enterprise environment mein security bahut tight hoti hai.

- **Inbound Rules:** Check karein ki kya RDP (Port 3389) ya SSH (Port 22) ke liye inbound rule allowed hai.
- **Priority Conflict:** Kabhi-kabhi koi high-priority rule (jaise `DenyAllInBound`) aapke allow rule ko block kar raha hota hai

2. Corporate Firewall & VPN Issues

Enterprise network mein traffic sirf Cloud se nahi, balki aapke office network se bhi control hota hai.

- **IP Whitelisting:** Ho sakta hai aapke office ka Public IP Azure NSG mein allow na ho.
- **VPN/ExpressRoute:** Agar aap VPN ya ExpressRoute use kar rahe hain, toh tunnel ke breakdown hone se connectivity toot sakti hai.

3. User-Defined Routes (UDR) & Forced Tunneling

Enterprise setups mein sara internet traffic aksar ek Central Firewall (jaise Azure Firewall) ke through bheja jata hai (Forced Tunneling).

- Agar UDR misconfigured hai, toh VM ka response rasta bhatak sakta hai (Asymmetric Routing), jiski wajah se connection fail ho jayega

4. OS Level Firewall & Services

VM ke andar ka Windows Firewall ya Linux `iptables` bhi connection block kar sakta hai.

- **Service Status:** Ho sakta hai VM boot toh ho gayi ho, lekin uske andar ki RDP/SSH service "Stopped" state mein ho.

5. Just-In-Time (JIT) Access

Kai companies **Microsoft Defender for Cloud** ka JIT access use karti hain. Isme ports tabhi khulte hain jab aap portal se access ki request karte hain. Agar request approved nahi hai, toh access nahi milega.

Isse Solve kaise karein?

Aap in tools ka use karke problem ka pata laga sakte hain:

1. **IP Flow Verify:** Yeh batayega ki kaunsa NSG rule traffic block kar raha hai.
2. **Serial Console:** Isse aap bina network ke direct VM ke andar login karke settings check kar sakte hain.
3. **Boot Diagnostics:** Isme aap VM ka screenshot dekh sakte hain ki kahi VM Blue Screen (BSOD) par toh nahi phasi